

Experiment - 7

Customer Segmentation using K-Means Clustering

```
In [1]: # Step 1: Load Dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv("Mall_Customers.csv")

print(df.head())

# Step 2: Explore Dataset

print("Dataset Shape:")
print(df.shape)

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Values:")
print(df.duplicated().sum())

# Step 3: Data Preprocessing

df = df.drop_duplicates()
df = df.dropna()

print("\nDataset after preprocessing:")
print(df.shape)

# Step 4: Select Relevant Features

X = df[[
    "Age",
    "Annual Income (k$)",
    "Spending Score (1-100)"
]]

print("\nSelected Features:")
print(X.head())
```

```
# Step 5: Feature Scaling

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\nScaled Data:")
print(X_scaled[:5])


# Step 6: Elbow Method

inertia = []

for k in range(2, 11):

    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    model.fit(X_scaled)

    inertia.append(model.inertia_)

plt.figure(figsize=(8, 5))

plt.plot(
    range(2, 11),
    inertia,
    marker="o"
)

plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.grid(True)

plt.show()


# Step 7: Silhouette Score

for k in range(2, 11):

    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(X_scaled)

    score = silhouette_score(
        X_scaled,
        labels
    )
```

```
print(
    "K =", k,
    "Silhouette Score =", round(score, 4)
)

# Step 8: Apply K-Means Clustering

k = 5

kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X_scaled)

print("\nClustered Data:")
print(df.head())

# Step 9: Calculate Cluster Centroids

centroids = scaler.inverse_transform(
    kmeans.cluster_centers_
)

centroids_df = pd.DataFrame(
    centroids,
    columns=[
        "Age",
        "Annual Income (k$)",
        "Spending Score (1-100)"
    ]
)

print("\nCluster Centroids:")
print(centroids_df)

# Step 10: Visualize Customer Segments

plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="Annual Income (k$)",
    y="Spending Score (1-100)",
    hue="Cluster",
    s=100
)

plt.scatter(
    centroids_df["Annual Income (k$)"],
    centroids_df["Spending Score (1-100)"],
    marker="x",
    s=250,
    c="black",
    label="Centroids"
```

```
)

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score")
plt.legend()

plt.show()

# Step 11: Analyze Customer Clusters

cluster_analysis = df.groupby("Cluster")[
    [
        "Age",
        "Annual Income (k$)",
        "Spending Score (1-100)"
    ]
].mean()

print("\nCluster Analysis:")
print(cluster_analysis.round(2))

# Step 12: Count Customers in Each Cluster

print("\nNumber of Customers in Each Cluster:")

print(
    df["Cluster"].value_counts().sort_index()
)

# Step 13: Save the Final Dataset

df.to_csv(
    "customer_segments.csv",
    index=False
)

print("\nCustomer segmentation completed successfully.")
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Dataset Shape:
(200, 5)

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 200 entries, 0 to 199

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	CustomerID	200 non-null	int64
1	Genre	200 non-null	object
2	Age	200 non-null	int64
3	Annual Income (k\$)	200 non-null	int64
4	Spending Score (1-100)	200 non-null	int64

dtypes: int64(4), object(1)

memory usage: 7.9+ KB

None

Statistical Summary:

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

Missing Values:

CustomerID	0
Genre	0
Age	0
Annual Income (k\$)	0
Spending Score (1-100)	0

dtype: int64

Duplicate Values:

0

Dataset after preprocessing:

(200, 5)

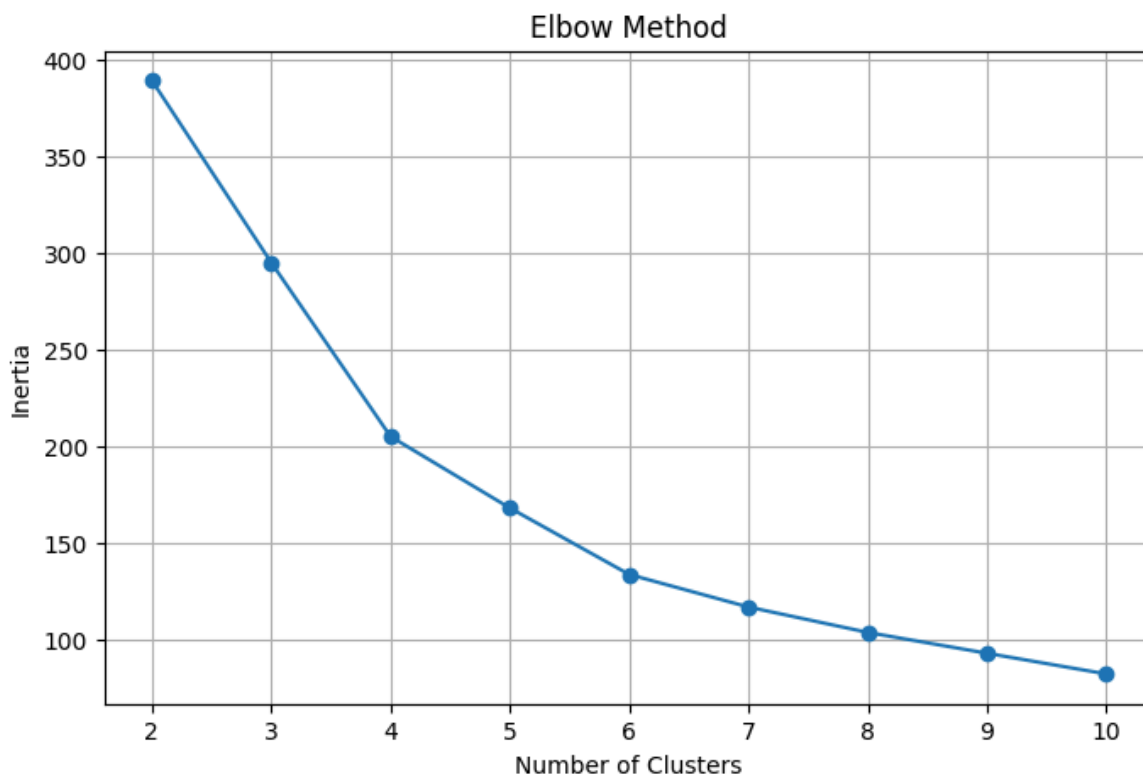
Selected Features:

	Age	Annual Income (k\$)	Spending Score (1-100)
0	19	15	39
1	21	15	81
2	20	16	6
3	23	16	77
4	31	17	40

Scaled Data:

```
[[-1.42456879 -1.73899919 -0.43480148]
 [-1.28103541 -1.73899919  1.19570407]]
```

```
[-1.3528021 -1.70082976 -1.71591298]
[-1.13750203 -1.70082976 1.04041783]
[-0.56336851 -1.66266033 -0.39597992]]
```



```
K = 2 Silhouette Score = 0.3355
K = 3 Silhouette Score = 0.3578
K = 4 Silhouette Score = 0.404
K = 5 Silhouette Score = 0.4166
K = 6 Silhouette Score = 0.4284
K = 7 Silhouette Score = 0.4172
K = 8 Silhouette Score = 0.4082
K = 9 Silhouette Score = 0.4177
K = 10 Silhouette Score = 0.4066
```

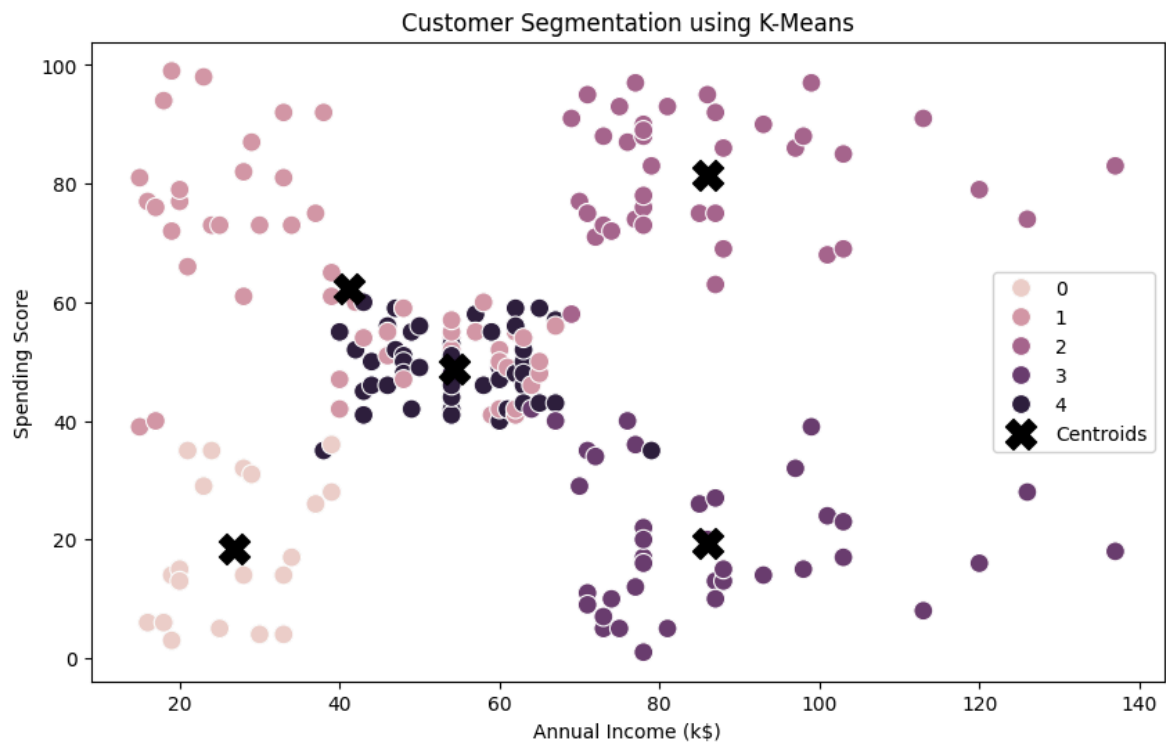
Clustered Data:

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)	\
0	1	Male	19	15	39	
1	2	Male	21	15	81	
2	3	Female	20	16	6	
3	4	Female	23	16	77	
4	5	Female	31	17	40	

	Cluster
0	1
1	1
2	0
3	1
4	1

Cluster Centroids:

	Age	Annual Income (k\$)	Spending Score (1-100)
0	46.250000	26.750000	18.350000
1	25.185185	41.092593	62.240741
2	32.875000	86.100000	81.525000
3	39.871795	86.102564	19.358974
4	55.638298	54.382979	48.851064



Cluster Analysis:

	Age	Annual Income (k\$)	Spending Score (1-100)
Cluster			
0	46.25	26.75	18.35
1	25.19	41.09	62.24
2	32.88	86.10	81.53
3	39.87	86.10	19.36
4	55.64	54.38	48.85

Number of Customers in Each Cluster:

Cluster	
0	20
1	54
2	40
3	39
4	47

Name: count, dtype: int64

Customer segmentation completed successfully.

Questions and Answers

1. What is clustering? How does it differ from classification?

Clustering is an unsupervised machine learning technique that groups similar data points into clusters based on their characteristics.

- **Clustering:** Works with unlabeled data and discovers natural groups.
- **Classification:** Works with labeled data and assigns data points to predefined classes.

2. Explain the working principle of the K-Means Clustering algorithm.

K-Means divides data into **K clusters** based on similarity.

Steps:

1. Select the number of clusters **K**.
2. Randomly initialize **K centroids**.
3. Assign each data point to the nearest centroid.
4. Calculate new centroids using the mean of the points in each cluster.
5. Repeat the assignment and centroid calculation until the centroids become stable or the stopping condition is reached.

3. Why is feature scaling important before applying K-Means clustering?

K-Means commonly uses **Euclidean distance** to measure similarity. If features have different scales, a feature with larger numerical values can dominate the distance calculation.

Feature scaling gives features comparable influence and can produce more meaningful clusters.

4. What is the Elbow Method, and how does it help determine the optimal number of clusters?

The **Elbow Method** runs K-Means for different values of **K** and calculates the within-cluster sum of squares (WCSS), also called inertia.

The WCSS generally decreases as K increases. The point where the decrease starts becoming much smaller is called the **elbow point**, which can be used as a practical choice for K.

5. What is the Silhouette Score? How is it used to evaluate clustering performance?

The **Silhouette Score** measures how well each data point fits within its assigned cluster compared with other clusters.

Its value ranges from **-1 to +1**:

- **Close to +1**: Data points are well separated and belong strongly to their clusters.
- **Around 0**: Clusters may overlap.
- **Close to -1**: Data points may have been assigned to an inappropriate cluster.

A higher score generally indicates better cluster separation and cohesion.

6. Why is K-Means considered an unsupervised machine learning algorithm?

K-Means is an **unsupervised learning algorithm** because it does not require predefined class labels. It analyzes the features of the data and automatically groups similar observations into clusters.

7. Mention any four real-world applications of customer segmentation.

1. **Targeted Marketing** – Sending suitable offers to different customer groups.
2. **Personalized Recommendations** – Recommending products based on customer behavior.
3. **Customer Retention** – Identifying customer groups that may need retention efforts.

4. **Product Planning** – Designing products according to the needs of different customer segments.

8. What are the limitations of the K-Means algorithm?

Some limitations are:

- The number of clusters **K** must be selected beforehand.
- Results can depend on the initial centroid positions.
- It can be sensitive to outliers.
- It works best when clusters are relatively compact and well separated.
- It may not perform well with clusters having irregular shapes or very different densities.
- Feature scaling can significantly affect the results.

9. How can businesses use customer segmentation to improve marketing and customer retention?

Businesses can group customers according to characteristics such as **purchasing behavior, spending, frequency, and preferences**.

They can then:

- Create targeted marketing campaigns.
- Provide personalized offers.
- Recommend relevant products.
- Identify high-value customers.
- Develop suitable customer-retention strategies.
- Allocate marketing resources more effectively.

10. Compare K-Means Clustering with Hierarchical Clustering based on their working principles and applications.

Feature	K-Means Clustering	Hierarchical Clustering
Working Principle	Divides data into K clusters using centroids	Builds a hierarchy of clusters
Number of Clusters	K must generally be specified	Can be selected from a dendrogram
Output	Separate clusters	Hierarchical tree structure
Speed	Generally faster for large datasets	Generally more computationally expensive
Applications	Customer segmentation, market analysis	Customer grouping, biological and document analysis
Visualization	Cluster plots	Dendrogram